Curso: construye tu app de ejercicios

Oct 27, 2026 · @Derek

Cómo funciona

Aprendes construyendo: cada fase termina con una pieza funcional de la app y un reto que resuelves tú antes de ver mi solución. El ritmo base es 5 h/semana, unas 19 semanas en total; se ajusta a tu agenda.

Ciclo de cada fase:

1. Yo explico el concepto y entrego solo el esqueleto (menos de una pantalla de código).
2. Tú implementas el reto en tu propio repositorio.
3. Me pasas el código y lo reviso: qué está bien, qué se rompe y por qué.
4. Cierras con el checkpoint: 3 a 5 preguntas que debes poder responder sin ayuda.

Como ya programas, el curso no repasa lógica básica: va directo a lo específico de la web, los datos locales y el dominio del entrenamiento.

Resultado final: una PWA en tu Android con planes por día, registro de series, récords, fatiga por músculo, tu historial importado y recomendaciones con IA, en código que entiendes y puedes modificar.

Mapa del curso

Son 12 fases; al terminar la fase 4 ya tienes una app que registra entrenamientos y reemplaza lo básico de Fit AI.

Fase	Tema	Entregable	Semanas
0	Entorno y herramientas	Repo en GitHub con proyecto vacío corriendo	0.5
1	JavaScript y TypeScript	Módulo con funciones de cálculo y pruebas	2
2	React e interfaz móvil	Pantallas navegables con datos de ejemplo	2
3	Datos locales	Base en el teléfono con ejercicios cargados	2
4	MVP de registro	Crear plan, entrenar y guardar series	2

Fase	Tema	Entregable	Semanas
5	PWA instalable y offline	App en la pantalla de inicio sin internet	1
6	Ciencia del entrenamiento	Funciones de 1RM, volumen y progresión	2
7	Récords, gráficas, mediciones	Historial con gráficas y récords automáticos	1.5
8	Fatiga muscular	Indicador de fatiga por músculo	2
9	Importación y respaldos	Historial de Fit AI dentro de la app	1
10	IA con LLM	Recomendaciones y planes generados	2
11	Android nativo (opcional)	APK con Capacitor	1

La fase 6 es teoría y se puede leer en paralelo a las fases 3 a 5.

Fase 0 · Entorno y herramientas

Al terminar tienes un proyecto vacío corriendo en tu computadora, abierto desde tu Android y respaldado en GitHub.

- VS Code, Node.js (versión LTS) y npm.
- Git: commit, rama y push; commits pequeños con mensaje claro.
- Estructura de un proyecto Vite con React y TypeScript.
- Abrir el servidor de desarrollo desde el celular, en la misma red Wi-Fi.

Reto: crear el repositorio `app-ejercicios`, levantar el proyecto con Vite y verlo en el navegador del teléfono.

Checkpoint: qué hace `npm run dev`; por qué conviene que los commits sean pequeños.

Fase 1 · JavaScript y TypeScript

Aprendes lo específico del lenguaje de toda la lógica de la app, y lo practicas con las fórmulas del entrenamiento.

- Objetos, arreglos y funciones flecha; `const` y `let`.
- `map`, `filter`, `reduce` y `sort`: reemplazan los ciclos en casi todo el código.
- Módulos (`import` y `export`) y organización de archivos.
- TypeScript: tipos, `interface`, uniones y genéricos básicos. Evita errores en datos como series y ejercicios.
- Asincronía: `Promise` y `async/await`, porque la base de datos local es asíncrona.

- Fechas: guardar en ISO 8601 y tener cuidado con las zonas horarias.
- Pruebas unitarias con Vitest.

Reto: escribir, con pruebas, `epley1RM(peso, reps)`, `volumenTotal(series)` y `agruparPorSemana(sesiones)`.

Checkpoint: diferencia entre `map` y `forEach`; qué devuelve una función `async`; por qué `0.1 + 0.2 !== 0.3` importa al sumar pesos.

Fase 2 · React e interfaz móvil

Construyes las pantallas de la app con datos de ejemplo; todavía sin base de datos.

- Componentes, props y estado (`useState`, `useEffect`); listas con `key`.
- Estado compartido con contexto o Zustand.
- Navegación entre pantallas con React Router y barra inferior.
- CSS para móvil: flexbox, grid, unidades relativas y áreas táctiles de al menos 44 px.
- Diseño de la pantalla de registro: peso y reps con una mano, botones grandes y teclado numérico (`inputMode="numeric"`).

Reto: construir Plan del día, Sesión activa e Historial leyendo de un archivo de datos de ejemplo.

Checkpoint: qué dispara un re-render; por qué una lista necesita `key`; cuándo usar estado local y cuándo compartido.

Fase 3 · Datos locales

Diseñas el modelo de datos y lo guardas en el teléfono con IndexedDB a través de Dexie; es la fase que más condiciona todo lo demás.

Entidades base:

- `Ejercicio`: id, nombre, músculo primario, músculos secundarios, equipo.
- `Plan` → `DiaPlan` → `EjercicioPlan`: series objetivo y rango de reps.
- `Sesion` (fecha, día del plan) → `Serie` (ejercicio, peso, reps, RPE).
- `Medicion`: fecha, tipo, valor.

Temas:

- Claves, índices y relaciones por id.
- Dexie: tablas, consultas, transacciones y migraciones de versión.
- Carga inicial de free-exercise-db convertida a tu esquema, más ejercicios propios.

- Capa de datos aislada: toda lectura y escritura pasa por un módulo `repository`, para poder cambiar de motor después.

Reto: cargar los ~800 ejercicios y hacer una búsqueda por nombre y músculo.

Checkpoint: por qué guardar ids y no nombres; qué es una migración y cuándo hace falta; qué ganas con el módulo `repository`.

Fase 4 · MVP de registro de entrenamientos

Cierras el primer ciclo completo: planeas un día, entrenas y guardas las series. Con esto ya reemplazas lo básico de Fit AI.

- Crear y editar un plan con días y ejercicios elegidos de la base.
- Sesión activa: peso y reps prellenados con los de la última vez, marcar serie completada y temporizador de descanso.
- Esfuerzo por serie con RPE o RIR.
- Editar y borrar series pasadas.
- Estado que sobrevive si cierras la app a media sesión.

Reto: registrar un entrenamiento real en la app. Lo que estorbe en el gimnasio se corrige antes de seguir.

Checkpoint: cuántos toques necesita registrar una serie; qué pasa si el teléfono se apaga a media sesión.

Fase 5 · PWA instalable y offline

Conviertes la web en una app instalable que funciona sin internet y tiene su respaldo de datos.

- Manifiesto web: nombre, íconos y `display: standalone`.
- Service worker y caché con el plugin PWA de Vite.
- Actualización de versiones sin romper los datos guardados.
- HTTPS y despliegue gratuito (Netlify o GitHub Pages).
- Almacenamiento persistente con `navigator.storage.persist()`, para que Android no lo borre.
- Botón de exportar y restaurar todo en JSON, porque limpiar los datos del sitio en Chrome borra la base.

Reto: desplegar, instalar en tu Android y registrar un entrenamiento en modo avión.

Checkpoint: qué hace el service worker; por qué una PWA exige HTTPS; qué pasa con tus datos si limpias el almacenamiento del navegador.

Fase 6 · Ciencia del entrenamiento aplicada

Pasas del registro a la decisión: qué carga poner la próxima vez. Son fórmulas y reglas conocidas, no magia, y aquí tu experiencia entrenando decide los parámetros.

- Esfuerzo: RPE y RIR, y su relación con la cercanía al fallo.
- 1RM estimado, válido sobre todo con 10 reps o menos (w = peso, r = reps):

```
\text{Epley: } 1RM = w\left(1 + \frac{r}{30}\right) \qquad \text{Brzycki: } 1RM = \frac{36\,w}{37 - r}
```

- Volumen: series efectivas por músculo por semana; 10 a 20 es un rango de partida común, que ajustas con tus resultados.
- Doble progresión: subes el peso cuando completas el tope del rango de reps en todas las series.
- Deload: bajar el volumen cada 4 a 6 semanas o tras varias sesiones con rendimiento a la baja.
- Toda regla de esta fase es heurística; sirve de guía, no de verdad absoluta.

Reto: escribir `siguienteCarga(historial, rangoReps)` con pruebas para cuatro casos: sube, mantiene, baja por fallo y deload.

Checkpoint: por qué Epley pierde precisión con muchas reps; cómo funciona la doble progresión; qué diferencia hay entre RPE y RIR.

Fase 7 · Récords, gráficas y mediciones

Conviertes el historial en información que se ve: récords que se detectan solos y gráficas de progreso.

- Récords: peso máximo, 1RM estimado y volumen máximo por sesión, detectados al guardar cada serie.
- Consultas agregadas en Dexie y series de tiempo.
- Gráficas con Recharts o Chart.js: 1RM por ejercicio en el tiempo y volumen semanal por músculo.
- Mediciones corporales (peso, cintura, brazo) con fecha y gráfica propia.
- Criterio de diseño: ejes rotulados con unidades, sin decoración.

Reto: pantalla de progreso por ejercicio con el 1RM estimado en el tiempo y los récords marcados.

Checkpoint: cuándo una serie cuenta como récord y cuándo no; por qué conviene calcular el 1RM al consultar y no guardarlo; qué agregación necesita el volumen semanal.

Fase 8 · Modelo de fatiga muscular

Modelas la fatiga de cada músculo como una carga que se acumula con cada serie y decae con el tiempo, igual que la respuesta de un sistema de primer orden.

- Carga de una serie: c = reps × peso × factor de esfuerzo (el factor sale del RPE).
- Reparto por músculo: 100% al primario y entre 30% y 50% a los secundarios.
- Decaimiento exponencial con vida media T½ de unas 48 a 72 h (t en horas, t_i = momento de la serie i):

```
F(t) = \sum_i c_i \, e^{-\ln 2 \,(t - t_i)/T_{1/2}}
```

- Normalización: F dividido entre tu capacidad, por ejemplo el percentil 90 de tu fatiga pico en las últimas semanas; se muestra como semáforo verde, amarillo o rojo.
- Calibración: comparas el indicador con cómo te sientes y ajustas T½ y los factores.
- Es una heurística: sirve como guía relativa entre músculos y días, no como medición fisiológica.

Reto: calcular el mapa de fatiga de tus últimas 2 semanas, con pruebas simples (una serie aislada baja a la mitad al pasar T½).

Checkpoint: qué pasa con F si entrenas el mismo músculo dos días seguidos; por qué normalizar; qué parámetro mueves si la app te marca descansado cuando sigues cansado.

Fase 9 · Importación y respaldos

Metes tu historial de Fit AI en la app y dejas listo el respaldo permanente.

- Formato de exportación propio en JSON, con número de versión.
- Lectura de CSV con Papa Parse y validación con Zod.
- Emparejar nombres de ejercicios: minúsculas, sin acentos, tabla de alias y revisión manual de los que no coinciden.
- Importación idempotente: importar dos veces no duplica sesiones.
- Si Fit AI no exporta: capturas de pantalla, yo extraigo los datos a CSV y tú los validas.

Reto: importar un CSV de prueba con 20 sesiones, dos veces seguidas, sin duplicados.

Checkpoint: cómo identificas que una serie ya fue importada; qué haces con un ejercicio que no existe en tu base; por qué versionar el formato de respaldo.

Fase 10 · IA con LLM

Añades recomendaciones y planes generados con una arquitectura híbrida: las reglas calculan y el modelo explica y propone.

- Qué calcula el código (cargas, fatiga, récords) y qué le toca al modelo (explicar, recomendar, armar planes).
- API de Claude: mensajes, instrucción de sistema y salida estructurada en JSON.
- Contexto resumido: últimas 4 a 8 semanas, volumen por músculo, récords y fatiga actual, no datos crudos.
- Validar la salida contra tu base de ejercicios con Zod; si el modelo inventa un ejercicio, se rechaza o se reintenta.
- La llave de la API no puede ir en el código de la PWA: va en una función serverless pequeña (Netlify Functions o Cloudflare Workers) con límite de uso y presupuesto.
- Límites: no sustituye a un profesional; dolor o lesión se derivan a un especialista.

Reto: botón "Armar plan de 4 días" que genera un plan válido y lo guarda como borrador.

Checkpoint: por qué la llave no va en el cliente; qué haces si el modelo devuelve JSON inválido; qué datos mandas y cuáles no.

Fase 11 · Android nativo con Capacitor (opcional)

Empaquetas la misma app en un APK solo si la PWA no cubre una necesidad concreta, como Health Connect o notificaciones más fiables.

- Qué es Capacitor y cómo agrega un proyecto Android a tu código web.
- Android Studio, APK de depuración e instalación por USB.
- Pasar el almacenamiento a SQLite nativo solo si hace falta; el módulo `repository` lo permite, y los datos migran con el respaldo JSON.
- Plugins útiles: Health Connect, notificaciones locales para el descanso y mantener la pantalla encendida.
- Actualizar el APK a mano cuando cambies la app.

Reto: instalar el APK en tu teléfono y restaurar tus datos desde el respaldo.

Checkpoint: qué funciones nativas justifican dar este paso; qué cambia en la capa de datos y qué no.

Reparto de trabajo

Yo explico y reviso; tú escribes el código y decides el producto.

Tarea	Tú	Claude
Explicar el concepto y dar el esqueleto		Sí
Implementar el reto de la fase	Sí	
Revisar tu código y explicar los errores		Sí
Decisiones de producto (pantallas, flujos, qué muestra la app)	Sí	Propongo opciones
Parámetros de entrenamiento (rangos de volumen, factores de fatiga)	Sí, con tu experiencia	Propongo valores iniciales
Probar en el gimnasio y reportar qué estorba	Sí	
Importar capturas de Fit AI a CSV	Validas	Sí

Cuando te atores más de 30 minutos en algo, me pasas el código y el mensaje de error completos; no hace falta que lo resuelvas a ciegas.

Recursos

Documentación oficial, buscable por título; no verifiqué cada enlace, así que ubícalos con el buscador.

- Fases 0 y 1: MDN Web Docs (guía de JavaScript), TypeScript Handbook, documentación de Vite y Vitest, Pro Git (libro gratuito).
- Fase 2: react.dev (tutorial oficial y "Learn React"), documentación de React Router.
- Fase 3: documentación de Dexie.js; repositorio free-exercise-db (GitHub, yuhonas) y el proyecto wger como segunda fuente.
- Fase 5: web.dev (sección Progressive Web Apps) y documentación de vite-plugin-pwa.
- Fases 6 a 8: artículos sobre doble progresión, RPE/RIR y volumen efectivo; revísalos con ojo crítico, porque las recomendaciones varían entre fuentes.
- Fase 9: documentación de Papa Parse y Zod.
- Fase 10: documentación de la API de Claude (docs.claude.com).
- Fase 11: documentación de Capacitor.